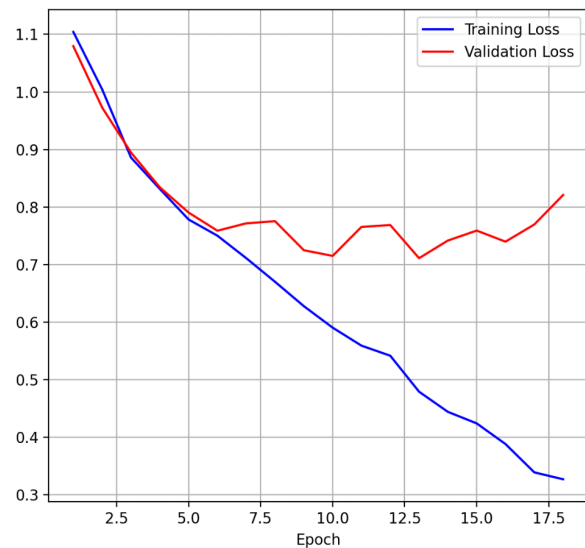
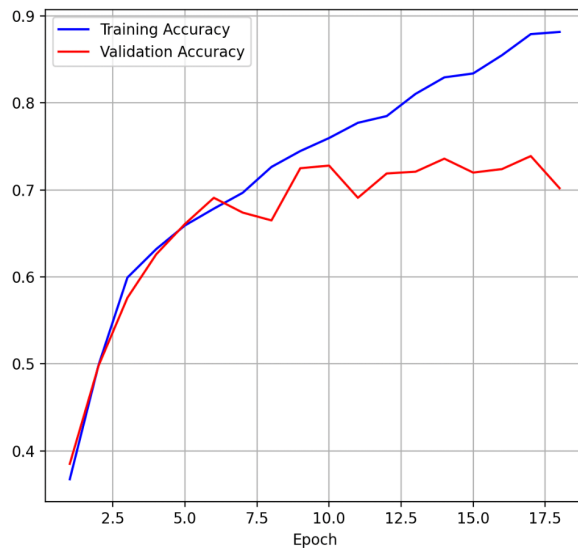


Hyper Parameter Optimization

Model: "sequential"		
Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 150, 150, 3)	0
conv2d (Conv2D)	(None, 148, 148, 16)	448
max_pooling2d (MaxPooling2D)	(None, 74, 74, 16)	0
dropout (Dropout)	(None, 74, 74, 16)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
conv2d_2 (Conv2D)	(None, 34, 34, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 64)	0
flatten (Flatten)	(None, 18496)	0
dense (Dense)	(None, 3)	55491
Total params: 79075 (308.89 KB)		
Trainable params: 79075 (308.89 KB)		
Non-trainable params: 0 (0.00 Byte)		

Loss and Accuracy Plots



Accuracy and Loss of Best Learned Model

```
* Evaluating basic_model
30/30 [=====] - 6s 212ms/step - loss: 0.7127 - accuracy: 0.7183
```

Hyper Optimization Strategy

The approach we took to optimize the model was primarily adjusting the convolutional layers. We also tested a few different approaches to using the dropout layers which involved testing a couple different drop rates and where it was positioned in the network. The most notable effect that we observed was the way in which the validation accuracy dips depending on the rate assigned to the dropout layer, which if assigned too high, created drastic shifts in the validation accuracy.